Estrategias y Pirámide de Automatización

Por qué no todo se automatiza por UI, el ROI en automatización y estrategias No-Code, Low-Code & High-Code

Hans Ortiz · Cristian Renz · Christopher Pallo

sofka_

www.sofka.com.co

Índice

01 Problema Común

02 La Pirámide de Automatizacion

03 Herrramientas & Frameworks

04 Estrategias de Automatización

05 Comparativa

06 Anti-Patrón - Alister Scott

07 Demo en Código

08 El Argumento Financiero

09 Decisión Estratégica

10 Matriz de Herramientas

11 Evolución del Concepto

12 El valor de automatizar ahora

13 Conclusiones

sofka_

El problema

¿Por qué fallan las estrategias de pruebas?

Tests Lentos

Suites de UI que tardan horas.
El equipo deja de ejecutarlas.

Tests Frágiles

Cambias un botón y se rompen
50 tests automatizados.

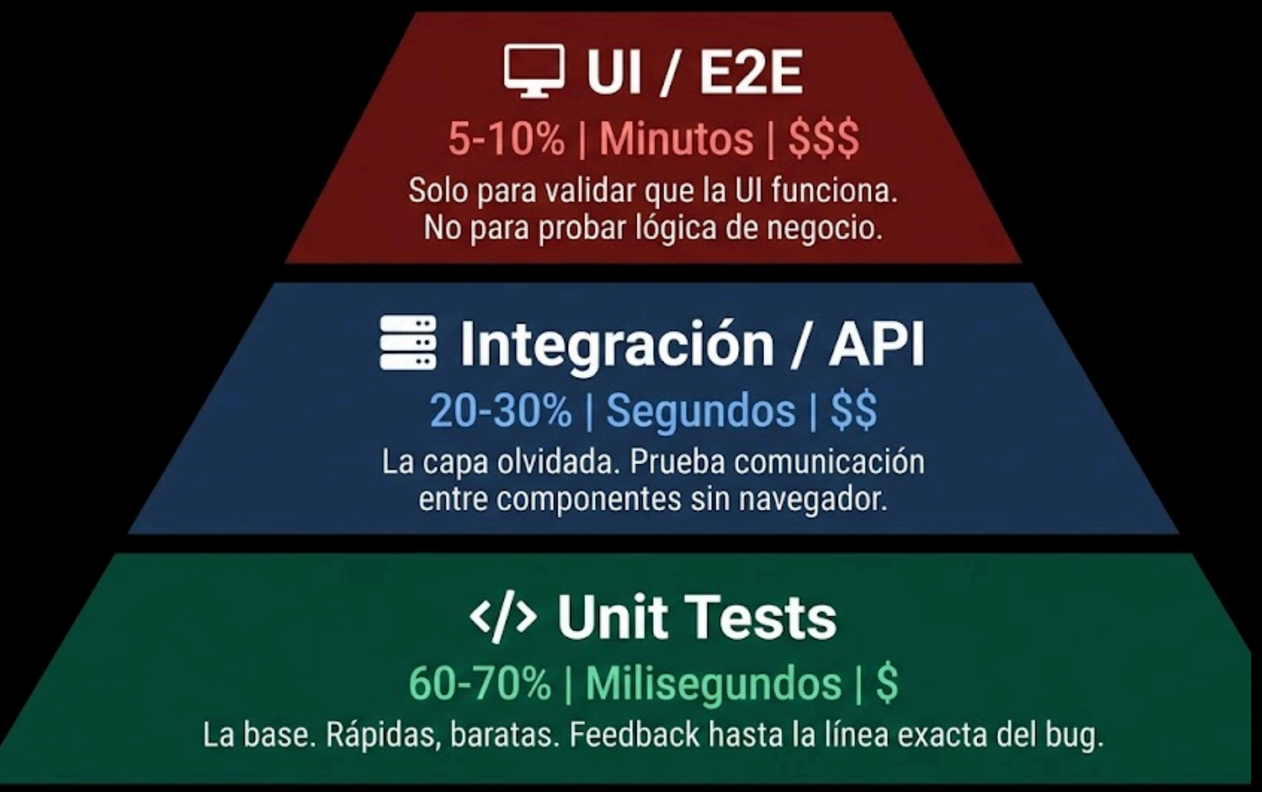
Costos Descontrolados

80% del presupuesto QA va
a mantener tests sin valor.

Feedback Tardío

'Algo falló' pero no se sabe
dónde exactamente.

Mike Cohn, 2009 — Dónde enfocar el esfuerzo



La Pirámide de Automatización

Principio Clave

Más pruebas rápidas y baratas en la base.
Menos pruebas lentas y costosas en la cima.

Unit Test	~5ms	Feedback instantáneo
API / Integración	~500ms	100x más lento
UI / Selenium	~30s	6,000x más lento

Herramientas & Frameworks

Ejemplos reales por cada nivel de la pirámide

UI / E2E	Integración / API	Unit Tests
Selenium WebDriver	REST Assured	JUnit 5 / TestNG
Cypress	Postman / Newman	Jest (JavaScript)
Playwright	Karate DSL	pytest (Python)
Appium (Mobile)	SuperTest (Node.js)	Mockito (Mocking)
TestCafe	WireMock (Mocking)	xUnit (.NET)

Estrategias de Automatización

No-Code ·Low-Code · High-Code

NO-CODE

Sin programación

Interfaz visual drag & drop. Ideal para QA manual en transición o validaciones rápidas de negocio.

HERRAMIENTAS

- Katalon Recorder
- Testim.io
- Mabl
- Leapwork

Ideal: QA Manual, POs, BAs

LOW-CODE

Código mínimo

Combina visual + scripts. Flexibilidad sin complejidad total. Balance entre accesibilidad y poder.

HERRAMIENTAS

- Katalon Studio
- Tosca
- Ranorex
- Robot Framework

Ideal: QA Semi-técnico, SDET Jr

HIGH-CODE

Programación completa

Máximo control y personalización. Frameworks profesionales para equipos de ingeniería QA.

HERRAMIENTAS

- Selenium + Java/Python
- Cypress + JS/TS
- Playwright + TS
- REST Assured + Java

Ideal: SDET, QA Automation Eng

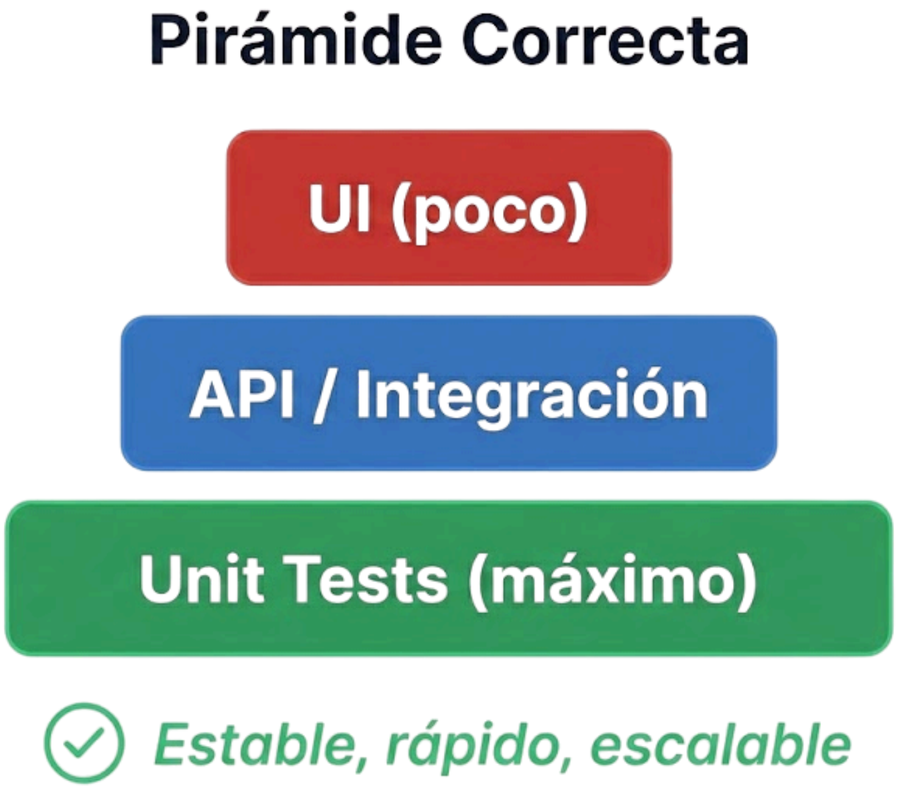
Comparativa

¿Cuándo usar cada estrategia?

Criterio	No-Code	Low-Code	High-Code
Curva aprendizaje	Baja (horas)	Media (días)	Alta (semanas)
Mantenibilidad	Baja	Media	Alta
Personalización	Limitada	Moderada	Total
Escalabilidad	Baja	Media	Alta
Integración CI/CD	Limitada	Buena	Nativa
Costo inicial	Bajo	Medio	Alto
Nivel pirámide	UI solamente	UI + API	Unit + API + UI

La mejor estrategia combina niveles: No-Code para validaciones rápidas, High-Code para la base de la pirámide.

Ice Cream Cone vs Pirámide



DEMO EN CÓDIGO — BUDGET MANAGEMENT APP

Mismo test, 3 niveles diferentes

UNIT TEST | ~5ms

JUnit 5

```java
@Test
void calcularBalance() {
  Cuenta c = new Cuenta();
  c.agregarIngreso(1000);
  c.agregarGasto(300);
  assertEquals(
    700, c.getBalance()
  );
}
```

API TEST | ~500ms

REST Assured

```java
@Test
void crearTransaccion() {
  Response res = given()
    .header("Auth",
      "Bearer " + token)
    .body("{monto: 150.50,
      tipo: gasto}")
    .post("/api/transacc");
  assertEquals(
    201, res.statusCode());
}
```

UI TEST | ~30s

Selenium

```java
@Test
void crearTransaccionUI(){
  driver.get("http://app");
  driver.findElement(
    By.id("email"))
    .sendKeys("user@t");
  driver.findElement(
    By.id("btnLogin"))
    .click();
  // Si cambia ID = ROMPE!
}
```

EL ARGUMENTO FINANCIERO

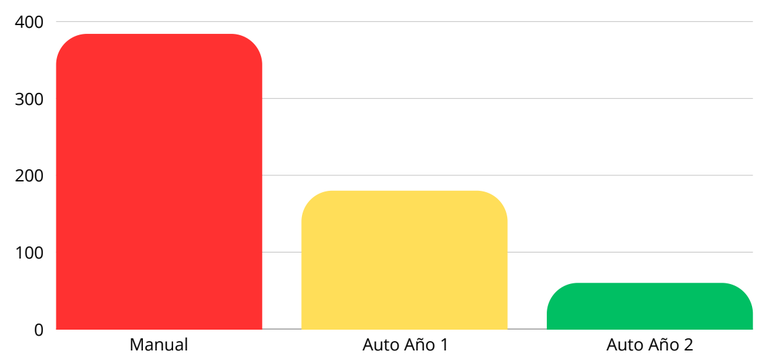
ROI en Automatización de Pruebas

Fórmulas Clave

ROI = (Beneficios - Costos) / Costos × 100%

Ahorro = (T.manual - T.auto) × N.tests × N.runs

Costos: Herramientas + scripts + capacitación + infra
Beneficios: Tiempo ahorrado + detección temprana + CI/CD



Ejemplo: Budget Management App

Casos regresión:	30
Ciclos/año:	24
Manual:	384h/año
Auto Año 1:	180h
Auto Año 2:	60h

Ley de Boehm

Un bug en producción cuesta

100x más

que detectarlo en desarrollo

DECISIÓN ESTRATÉGICA

¿Qué automatizar y qué no?

SÍ AUTOMATIZAR

- Smoke tests y regression (cada build)
- Tests data-driven (mismo flujo, N datos)
- Tests de API / servicios (alto ROI)
- Tests de performance y carga
- Cross-browser y cross-platform
- Validaciones lógicas y matemáticas
- Flujos críticos (login, pagos, CRUD)

NO AUTOMATIZAR

- Testing exploratorio (creatividad humana)
- Pruebas UX / usabilidad (juicio subjetivo)
- Tests que se ejecutan una sola vez
- Features inestables o en desarrollo
- Validación visual subjetiva
- Tests con expertise de dominio única
- Tests con resultados no predecibles

MATRIZ DE HERRAMIENTAS

Frameworks por nivel y estrategia de automatización

Nivel	No-Code	Low-Code	High-Code
UI / E2E	Katalon Recorder Testim.io Mabl	Katalon Studio Ranorex TestComplete	Selenium + Java Cypress + JS Playwright + TS
API / Integración	Postman (GUI) SoapUI Insomnia	Karate DSL Robot Framework Tosca API	REST Assured SuperTest HttpClient + JUnit
Unit	–	Robot Framework (Keywords)	JUnit 5 / TestNG Jest / Vitest pytest / Mockito

→ La tendencia del mercado: equipos maduros combinan High-Code para Unit + API y Low-Code para flujos E2E complejos.

Más allá de la pirámide: modelos modernos

Testing Trophy

Kent C. Dodds

Mayor énfasis en tests de integración.
Mejor balance confianza vs velocidad.

```
Static > Unit > Integration >
E2E
```

Testing Honeycomb

Spotify

Diseñado para microservicios.
Énfasis en integración entre servicios.

```
Ideal para arquitecturas
distribuidas
```

Testing Diamond

Comunidad 2024

Pocas unitarias, muchas integración, pocas E2E. Refleja arquitecturas actuales.

```
Balance pragmático
```

Por qué cada Sprint cuenta

01

Automatizar en el Sprint

El beneficio máximo se obtiene cuando la prueba se crea junto con la funcionalidad. El diseño del test influye en el diseño del producto.

Máximo ROI

02

Automatizar después

El costo sube drásticamente (curva en S). Se pierde la oportunidad de influir en el diseño y la deuda técnica crece.

Costo +300%

03

No automatizar

Pruebas manuales cada Sprint son 'prohibitivamente caras'. La suite no crece, el equipo se desborda con regresiones.

Insostenible

Liberar el talento humano

Al automatizar lo repetitivo, el equipo puede enfocarse en pruebas exploratorias: descubrir errores inesperados, casos no previstos e ideas que faltan en las historias de usuario — algo que una máquina no puede hacer.

Conclusiones

5 puntos clave para recordar

1

La pirámide es un modelo mental para distribuir esfuerzo, no una regla rígida.

2

Automatizar por UI es frágil, lento y costoso. Prueba la lógica donde vive.

3

No-Code para validar rápido, Low-Code para flexibilidad, High-Code para la base.

4

El ROI guía las decisiones: automatiza lo que da mayor retorno en el mismo Sprint.

5

Elige herramientas por nivel: JUnit/Jest para unit, REST Assured para API, Playwright para E2E.

¡Gracias!

Better
happens.
Every day.

Medellín
Calle 12 # 30 – 80

Bogotá
Centro Empresarial Paralelo 108 Av. Carrera 45 No. 108-27 Torre 2 Oficina 1103

México
Av. Paseo de la Reforma 284

USA Miami, FL
100 Southeast Third Avenue Suite 1000 Fort Lauderdale, FL 33394

Ecuador
Blvr. Naciones Unidas, Quito 170135, Ecuador - Edificio Metropolitan

Uruguay
Paysandú, Salto 916

sofka_

www.sofka.com.co